

CICLO FORMATIVO DE GRADO SUPERIOR
DESARROLLO DE APLICACIONES
MULTIPLATAFORMA

Sistema de Gestión de Academia

Trabajo de Fin de Grado (TFG)

Aplicación web para la gestión integral de alumnos, profesores, cursos, matrículas y pagos de una academia de formación

José Manuel Román

2.º DAM — Desarrollo de Aplicaciones Multiplataforma

Mayo de 2025

Resumen

Este Trabajo de Fin de Grado consiste en el desarrollo de una aplicación web completa para la gestión de una academia de formación. La aplicación permite administrar de forma centralizada los alumnos, profesores, cursos, matrículas y pagos del centro.

El sistema se compone de un backend desarrollado con **Spring Boot** (Java 21) que expone una API REST, y un frontend construido con **HTML, CSS y JavaScript puro** sin ningún framework adicional. La autenticación se basa en **JWT (JSON Web Tokens)** y el control de acceso distingue tres roles: administrador, profesor y alumno.

La base de datos es **PostgreSQL**, alojada en la nube mediante el servicio Neon, y el esquema se genera automáticamente a través de Hibernate. El proyecto incluye pruebas unitarias con JUnit 5.

Palabras clave

Spring Boot, Java, JWT, REST API, PostgreSQL, JavaScript vanilla, gestión académica, DAM, TFG.

Enlace al Código

Github : <https://github.com/Chema-rgb/TFG-DAM2>

Índice

1. Introducción y motivación

2. Objetivos del proyecto

3. Planificación y metodología

4. Análisis de requisitos

5. Diseño del sistema

6. Diseño de la base de datos

7. Implementación del backend

8. Implementación del frontend

9. Seguridad

10. Pruebas

11. Manual de usuario

12. Despliegue

13. Conclusiones y trabajo futuro

14. Bibliografía

1. Introducción y motivación

En el contexto actual, donde la digitalización de procesos administrativos es fundamental para la competitividad de cualquier organización, muchas academias de formación siguen gestionando sus datos de forma manual o con herramientas poco integradas como hojas de cálculo. Esto provoca errores, duplicidades de información y dificultades para obtener una visión global del estado del centro.

Este proyecto surge de la necesidad real de centralizar y automatizar la gestión de una academia de formación. El objetivo es proporcionar una herramienta web accesible desde cualquier dispositivo con navegador, sin necesidad de instalar ningún software adicional en el equipo del usuario.

La idea es sencilla: que el administrador del centro pueda, desde un único lugar, dar de alta alumnos y profesores, crear cursos, gestionar las matrículas y llevar un control de los pagos. Al mismo tiempo, los profesores pueden consultar la información que les es relevante y los alumnos acceden a su área personal.

1.1 Contexto tecnológico

Las aplicaciones web basadas en arquitectura cliente-servidor con API REST se han convertido en el estándar de facto para el desarrollo de software empresarial. Este modelo permite separar claramente la lógica de negocio (backend) de la presentación (frontend), lo que facilita el mantenimiento, las pruebas y la escalabilidad.

Spring Boot es uno de los frameworks más utilizados en el ecosistema Java para construir backends REST. Su sistema de autoconfiguración reduce enormemente el código repetitivo y permite centrarse en la lógica de la aplicación. Combinado con Spring Security y JWT, proporciona un sistema de autenticación robusto y sin estado.

En el lado del cliente, se ha optado por JavaScript puro en lugar de frameworks como React o Angular. Esta decisión tiene dos motivos principales: primero, demostrar que se puede construir una interfaz funcional y mantenible sin depender de frameworks externos; segundo, que el aprendizaje y uso de JavaScript puro refuerza la comprensión de los fundamentos del lenguaje.

1.2 Alcance del proyecto

El sistema cubre las siguientes áreas funcionales:

- **Gestión de alumnos:** permite dar de alta, consultar, modificar y eliminar alumnos del sistema.
- **Gestión de profesores:** cubre las mismas operaciones básicas aplicadas a los profesores.
- **Gestión de cursos:** incluye la creación de cursos, la asignación de un profesor responsable, y el control tanto del aforo como del estado del curso.
- **Gestión de matrículas:** permite inscribir alumnos en los cursos disponibles, comprobando en todo momento las plazas libres.
- **Gestión de pagos:** registra los cobros asociados a cada matrícula y lleva un seguimiento de su estado, que puede ser pendiente, pagado, vencido o cancelado.
- **Panel de administración:** ofrece una visión general del sistema mediante estadísticas, gestión de usuarios y análisis del nivel de ocupación de los cursos.
- **Panel Alumno:** el alumno autenticado puede consultar su perfil personal en modo solo lectura.
- **Panel Profesor:** el profesor que se autentifique puede consultar alumnos, matrículas y cursos en modo solo lectura.

2. Objetivos del proyecto

2.1 Objetivo general

Desarrollar una aplicación web funcional para la gestión integral de una academia de formación, que cubra desde la autenticación de usuarios hasta la gestión de pagos, con control de acceso basado en roles.

2.2 Objetivos específicos

1. Diseñar e implementar una API REST con Spring Boot que cubra las operaciones CRUD necesarias para las entidades del sistema.
2. Implementar autenticación mediante JWT, con expiración configurable y validación de firma.
3. Definir un sistema de autorización basado en roles (ADMIN, PROFESOR y ALUMNO) que limite el acceso a los endpoints en función del usuario autenticado.
4. Desarrollar un frontend responsive con JavaScript puro que consuma la API y ofrezca una interfaz sencilla e intuitiva.
5. Almacenar los datos en una base de datos PostgreSQL con un esquema bien definido y relaciones coherentes entre entidades.
6. Aplicar validaciones tanto en el backend (unicidad de DNI, precios no negativos, control de aforo) como en el frontend (campos obligatorios y respuesta visual al usuario).
7. Escribir pruebas unitarias para cubrir la lógica de negocio más relevante.
8. Desplegar el backend en la nube junto con la base de datos, utilizando Neon PostgreSQL como servicio cloud.

2.3 Requisitos no funcionales

Requisito	Descripción
Usabilidad	Interfaz clara, con mensajes de error y éxito visibles. Formularios en modales para no perder el contexto.
Seguridad	Contraseñas hasheadas con BCrypt. Tokens JWT. Endpoints protegidos por rol.
Rendimiento	Filtrados en local para minimizar llamadas al backend. Respuestas JSON ligeras.
Mantenibilidad	Código modular: un fichero JS por funcionalidad. Funciones compartidas en "funciones.js"
Portabilidad	El frontend solo requiere un servidor HTTP estático. El backend se empaqueta como JAR ejecutable.

3. Planificación y metodología

3.1 Metodología de desarrollo

El desarrollo del proyecto se llevó a cabo de manera iterativa e incremental, adoptando una metodología ágil. Esto implica que, en lugar de establecer una planificación exhaustiva desde el inicio, el trabajo se organizó en sucesivas iteraciones, cada una de las cuales incorporaba nueva funcionalidad sobre la base desarrollada anteriormente.

- **Iteración 1:** Configuración del proyecto, modelo de datos, autenticación JWT.
- **Iteración 2:** CRUD de alumnos y profesores con validaciones.
- **Iteración 3:** Gestión de cursos y matrículas con control de aforo.
- **Iteración 4:** Módulo de pagos y panel de administración con estadísticas.
- **Iteración 5:** Frontend completo, pruebas y documentación.

Gracias a este enfoque permitió identificar y resolver problemas de forma temprana, facilitando los ajustes necesarios a lo largo del proceso y evitando que los errores se acumularan hasta las fases finales del desarrollo.

3.2 Herramientas utilizadas

Herramienta	Propósito
VS Code	Entorno de desarrollo (IDE)
Maven	Gestión de dependencias y build del backend
GitHub	Control de versiones y repositorio remoto
Postman	Pruebas manuales de los endpoints REST
Neon	Base de datos PostgreSQL en la nube
Swagger/OpenAPI	Documentación automática de la API
DBBeaver	Explorar la base de datos

3.3 Planificación temporal

Semana	Tarea
1–2	Análisis de requisitos, diseño de la base de datos y Figma, setup del Spring Boot
3–4	Implementación de entidades, repositorios y autenticación JWT
5–6	Controllers de alumnos, profesores y cursos y algunas validaciones
7–8	Matrículas, pagos y panel de administración, además de la continuación controllers
9–10	Frontend: estructura HTML, CSS y módulos JavaScript
11–12	Integración frontend–backend, pruebas de funcionamiento y corrección de errores
13–14	Pruebas unitarias, documentación, memoria y pequeñas implementaciones extra

4. Análisis de requisitos

4.1 Actores del sistema

El sistema define tres tipos de usuario con distintos permisos:

- **Administrador:** tiene acceso completo a la aplicación. Puede crear, modificar y eliminar cualquier entidad del sistema (alumnos, profesores, cursos, matrículas y pagos), y con acceso a un panel con estadísticas generales del centro.
- **Profesor:** puede consultar la información de alumnos, cursos y matrículas relacionados con su actividad docente, pero no tiene permisos para modificar datos de otros usuarios ni realizar operaciones de escritura fuera de su ámbito.
- **Alumno:** accede únicamente a su propio perfil, donde puede consultar sus datos. No tiene visibilidad sobre el resto de usuarios del sistema.

4.2 Casos de uso principales

CU-01: Iniciar sesión

Campo	Descripción
Actor	Cualquier usuario
Precondición	El usuario tiene una cuenta activa en el sistema
Flujo normal	El usuario introduce su nombre de usuario y contraseña. El sistema valida las credenciales y devuelve un token JWT. El usuario es redirigido al dashboard.
Flujo alternativo	Si las credenciales son incorrectas, se muestra un mensaje de error y no se genera token.

CU-02: Gestión de alumnos

Campo	Descripción
Actor	ADMIN (escritura), PROFESOR (solo lectura)
Flujo normal	El admin accede a la lista de alumnos, puede crear uno nuevo, editarlo o borrarlo, afectando a su cuenta también. El profesor solo puede consultar la lista.
Restricciones	DNI único en el sistema. Teléfono único. Nombre y apellidos obligatorios.

CU-03: Matricular alumno en curso

Campo	Descripción
Actor	ADMIN
Precondición	El alumno y el curso existen y están activos
Flujo normal	El admin selecciona alumno y curso, indica el estado y confirma. El sistema crea la matrícula.
Flujo alternativo 1	Si el curso está lleno, el sistema devuelve HTTP 409 con código CURSO_COMPLETO y el frontend muestra un modal de aviso.
Flujo alternativo 2	Si el alumno ya está matriculado en ese curso, el sistema devuelve error de restricción única.

CU-04: Registrar pago

Campo	Descripción
Actor	ADMIN
Flujo normal	El admin crea un registro de pago asociando un alumno, opcionalmente un curso, un concepto, importe y fechas. El estado inicial puede ser PENDIENTE o PAGADO.
Restricciones	El curso es opcional (un pago puede ser por conceptos no ligados a un curso específico).

CU-05: Consultar estadísticas

Campo	Descripción
Actor	ADMIN
Flujo normal	El panel de administración muestra: total recaudado, pagos pendientes, pagos vencidos, alumnos con pagos vencidos, porcentaje de ocupación por curso.

CU-06: Consultar mi perfil

Campo	Descripción
Actor	ALUMNO
Precondición	El alumno ha iniciado sesión y dispone de un token JWT válido
Flujo normal	El alumno accede a «Mi perfil». El frontend realiza GET /api/alumnos/me con el token JWT. El sistema extrae el username del token, localiza el alumno vinculado a ese usuario y devuelve sus datos personales. El alumno los consulta en modo solo lectura.
Flujo alternativo 1	Si el token es inválido o ha expirado, el sistema devuelve HTTP 401 y el frontend redirige al login.

4.3 Requisitos funcionales

ID	Requisito
01	El sistema debe autenticar usuarios mediante usuario y contraseña, devolviendo un JWT
02	El token JWT debe incluir el nombre de usuario y el rol
03	El sistema debe rechazar peticiones sin token o con token expirado con HTTP 401
04	El CRUD de alumnos debe validar unicidad de DNI y teléfono
05	El CRUD de profesores debe validar unicidad de DNI y teléfono
06	Los cursos deben tener precio no negativo
07	Al matricular, el sistema debe verificar que no se supere la capacidad del curso
08	No puede existir más de una matrícula activa del mismo alumno en el mismo curso
09	Al borrar un alumno, sus matrículas y pagos deben eliminarse en cascada, además de sus cuenta vinculada
10	Al borrar un profesor, sus cursos deben quedarse sin profesor asignado (no borrar cursos)
11	El panel de admin debe mostrar el total recaudado y los pagos en distintos estados
12	Un administrador no puede eliminarse a sí mismo desde el panel de usuarios
13	Al crear un alumno, el sistema debe generar automáticamente su cuenta de usuario con username=DNI, password=DNI
14	Todas las páginas HTML deben incluir el favicon SVG de la aplicación.

5. Diseño del sistema

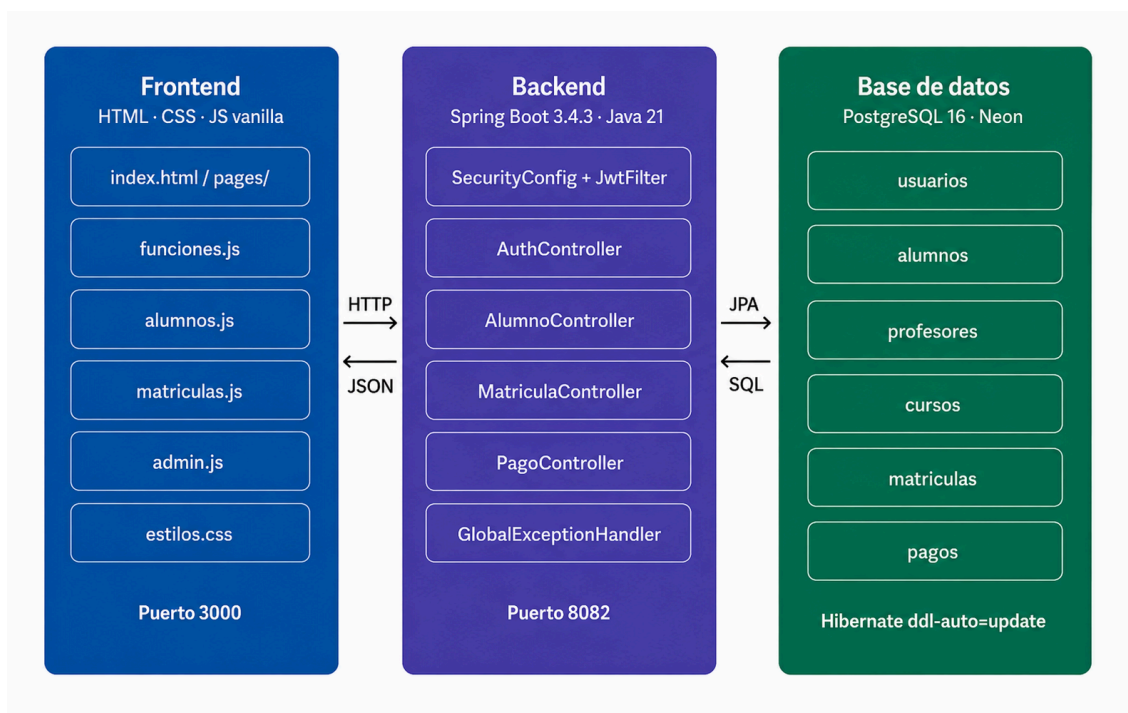
5.1 Arquitectura general

El sistema sigue una arquitectura de tres capas bien diferenciadas:

- **Capa de presentación** (Frontend): HTML + CSS + JavaScript puro
- **Capa de lógica de negocio** (Backend): API REST construida con Spring Boot.
- **Capa de datos** (Base de datos): PostgreSQL gestionado mediante JPA/Hibernate.

La comunicación entre el frontend y el backend se realiza a través de HTTP usando el formato JSON. El frontend nunca accede directamente a la base de datos.

Imagen de la arquitectura de capas



5.2 Arquitectura del backend

El backend sigue el patrón estándar de Spring Boot en capas:

Capa	Paquete	Responsabilidad
Controladores	controller	Recibir peticiones HTTP, delegar lógica, devolver respuestas
Servicios	service	Lógica de negocio de usuarios (en este proyecto la mayor parte está en los controllers)
Repositorios	repository	Acceso a datos mediante JPA
Modelos	model	Entidades JPA que mapean las tablas de la base de datos
Seguridad	security	Filtro JWT y utilidades de token
Configuración	config	Configuración de seguridad, CORS y manejo global de excepciones
DTOs	dto	Objetos de transferencia para login

Nota: en este proyecto la lógica de negocio reside en los propios controllers, sin una capa de servicios intermedia. Esto simplifica la estructura para el alcance del proyecto, aunque en una aplicación de mayor envergadura se separaría en una capa service dedicada. Se reconoce como deuda técnica en el apartado de trabajo futuro.

5.3 Estructura de directorios del proyecto

La estructura del proyecto se divide en dos módulos principales: backend y frontend.

Backend

El backend sigue la organización estándar de un proyecto Maven, con el descriptor **pom.xml** en la raíz y el código fuente ubicado bajo `src/main/java/com/academia/`. Desde la clase de arranque **AcademiaApplication.java** se ramifica el resto del código en paquetes según su responsabilidad:

- **config**: configuración de seguridad y manejador global de excepciones
- **controller**: controladores REST de cada recurso de la aplicación (alumnos, cursos, matrículas, pagos, profesores, usuarios y autenticación)
- **model**: entidades JPA que representan el modelo de datos del dominio.
- **repository**: interfaces de acceso a base de datos para cada entidad.
- **service**: lógica de negocio asociada a los usuarios.
- **security**: componentes del sistema de autenticación JWT, concretamente `JwtFilter.java` y `JwtUtil.java`.
- **dto**: objetos de transferencia de datos empleados en el proceso de login.
- **resources/application.properties**: fichero central de configuración de la aplicación.
- **test/**: pruebas unitarias sobre la entidad alumno y sobre la utilidad JWT.

Frontend

El frontend tiene una estructura sencilla, acorde con el enfoque de HTML estático adoptado:

- **index.html**: punto de entrada de la aplicación (pantalla de login).
- **favicon.svg**: icono SVG de la aplicación, incluido en todas las páginas HTML mediante la etiqueta `<link rel="icon">`.
- **css/estilos.css**: hoja de estilos global.
- **js/funciones.js**: utilidades JavaScript compartidas entre vistas (`llamarApi`, `verificarAcceso`, `construirMenu`, `mostrarExito`).
- **js/panel.js**, **js/alumnos.js**, **js/profesores.js**, **js/mi-perfil.js**: lógica específica de cada sección de la interfaz.
- **pages/**: vistas HTML de cada sección: `admin.html`, `alumnos.html`, `cursos.html`, `dashboard.html`, `matricula.html`, `pagos_lista.html`, `profesores.html` y `mi-perfil.html`.

5.4 Tecnologías y versiones

Tecnología	Versión	Uso
Java	21 (LTS)	Lenguaje del backend
Spring Boot	3.4.3	Framework backend
Spring Security	6.x (incluido en Boot)	Autenticación y autorización
JJWT	0.12.6	Generación y validación de tokens JWT
Spring Data JPA	3.x	ORM sobre Hibernate
PostgreSQL Driver	42.x	Conexión con la base de datos
Lombok	1.18.x	Generación de getters, setters, constructores
SpringDoc OpenAPI	2.3.0	Documentación Swagger automática
Maven	3.9.x	Gestión de dependencias y build
PostgreSQL (Neon)	16	Base de datos en la nube
JavaScript	ES2022 (vanilla)	Lógica del frontend
HTML5 / CSS3	—	Estructura y estilos del frontend

6. Diseño de la base de datos

6.1 Modelo entidad-relación

La base de datos está compuesta por seis tablas principales. A continuación se describe cada entidad y sus relaciones:

- **usuarios**: cuenta de acceso al sistema con rol.
- **alumnos**: datos personales del estudiante, vinculado a usuarios (1:1).
- **profesores**: datos del docente, vinculado a usuarios (1:1).
- **cursos**: información del curso, con referencia opcional al profesor (N:1).
- **matriculas**: relación N:M entre alumnos y cursos.
- **pagos**: registros de cobro asociados a un alumno y opcionalmente a un curso.

6.2 Descripción de tablas

Tabla: usuarios

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
username	VARCHAR	UNIQUE, NOT NULL	Nombre de usuario para login
password	VARCHAR	NOT NULL	Contraseña encriptada con BCrypt
email	VARCHAR	UNIQUE	Correo electrónico
rol	VARCHAR	NOT NULL	ADMIN / PROFESOR / ALUMNO
activo	BOOLEAN	DEFAULT true	Si la cuenta está activa
fecha_creacion	TIMESTAMP		Fecha de alta en el sistema

Tabla: alumnos

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
nombre	VARCHAR	NOT NULL	Nombre del alumno
apellidos	VARCHAR	NOT NULL	Apellidos del alumno
dni	VARCHAR	UNIQUE,NOT NULL	Documento de identidad
telefono	VARCHAR	UNIQUE	Teléfono de contacto
email	VARCHAR	NOT NULL	Correo del alumno
direccion	VARCHAR		Dirección postal
fecha_nacimiento	DATE		Fecha de nacimiento
fecha_inscripcion	DATE		Fecha en que se registró
estado	VARCHAR		Activo / Inactivo / Baja
usuario_id	BIGINT	FK → usuarios,NOT NULL	Cuenta de acceso asociada

Tabla: profesores

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
nombre	VARCHAR	NOT NULL	Nombre del profesor
apellidos	VARCHAR	NOT NULL	Apellidos del profesor
dni	VARCHAR	UNIQUE	Documento de identidad
telefono	VARCHAR	UNIQUE	Teléfono de contacto
email	VARCHAR		Correo del profesor
especialidad	VARCHAR		Materia o especialidad que imparte
estado	VARCHAR		ACTIVO / INACTIVO
usuario_id	BIGINT	FK → usuarios	Cuenta de acceso asociada

Tabla: cursos

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
nombre	VARCHAR	NOT NULL	Nombre del curso
descripcion	TEXT		Descripción del contenido
nivel	VARCHAR		Nivel: Básico, Intermedio, Avanzado...
capacidad	INTEGER		Número máximo de alumnos
precio	DECIMAL	≥ 0	Precio de matrícula en euros
estado	VARCHAR		Activo / Inactivo / Completo
profesor_id	BIGINT	FK → profesores, nullable	Profesor asignado al curso

Tabla: matriculas

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
alumno_id	BIGINT	FK → alumnos, NOT NULL	Alumno matriculado
curso_id	BIGINT	FK → cursos, NOT NULL	Curso en que se matricula
fecha_matricula	DATE		Fecha de la matrícula
estado	VARCHAR		Activa / Cancelada / Completada

NOTA - Restricción UNIQUE sobre (alumno_id, curso_id) para evitar duplicados

Tabla: pagos

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
alumno_id	BIGINT	FK → alumnos, NOT NULL	Alumno que paga
curso_id	BIGINT	FK → cursos, nullable	Curso relacionado (opcional)
concepto	VARCHAR		Descripción del pago
importe	DECIMAL		Cantidad en euros
fecha_pago	DATE		Fecha en que se realizó el pago
fecha_vencimiento	DATE		Fecha límite para pagar
estado	VARCHAR		Pendiente / Pagado / Vencido / Cancelado
observaciones	TEXT		Notas adicionales

6.3 Relaciones entre entidades

La relación que presenta las entidades es la siguiente:

- **usuarios** → **alumnos**: relación 1:1 obligatoria. Ya que al eliminar un alumno se debe eliminar obligatoriamente su usuario
- **usuarios** → **profesores**: relación 1:1 (opcional)
- **profesores** → **cursos**: relación 1:N (un profesor puede dar varios cursos; un curso tiene como máximo un profesor).
- **alumnos** → **matriculas** → **cursos**: relación N:M mediante tabla intermedia.
- **alumnos** → **pagos**: relación 1:N (un alumno puede tener varios pagos).
- **cursos** → **pagos**: relación 1:N (opcional).

7. Implementación del backend

7.1 Configuración inicial — AcademiaApplication.java

El punto de entrada de la aplicación se extiende en `SpringApplication` y, al arrancar `SpringApplication` comprueba si existe ya un usuario administrador en la base de datos. Si no existe, lo crea con las credenciales por defecto:

```
@SpringBootApplication
public class AcademiaApplication {    Can't initialize javac processor due to (mos

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        SpringApplication.run(AcademiaApplication.class, args);
    }

    @Bean
    CommandLineRunner inicializarDatos(UsuarioRepository usuarioRepository, Passwo

        return args -> {
            // creo el admin por defecto si no existe todavía
            if (!usuarioRepository.existsByUsername(username: "admin")) {
                Usuario admin = new Usuario();
                admin.setUsername(username: "admin");
                admin.setPassword(passwordEncoder.encode("admin123"));
                admin.setEmail(email: "admin@academia.com");
                admin.setRol(Usuario.Rol.ADMIN);
                admin.setActivo(activo: true);
                usuarioRepository.save(admin);
                System.out.println("Usuario admin creado: admin / admin123");
            }
        };
    }
}
```

7.2 Modelos (entidades JPA)

Cada entidad usa las anotaciones de JPA para mapear con la tabla correspondiente de PostgreSQL. Hibernate genera y actualiza el esquema automáticamente

Ejemplo: Matricula.java

```
@Entity
@Table(name = "matriculas", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"alumno_id", "curso_id"})
})
public class Matricula {    Can't initialize javac processor due

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "alumno_id", nullable = false)
    private Alumno alumno;

    @ManyToOne
    @JoinColumn(name = "curso_id", nullable = false)
    private Curso curso;

    @Column(name = "fecha_matricula")
    private LocalDate fechaMatricula;

    @Enumerated(EnumType.STRING)
    private Estado estado;

    public Matricula() {}
```

Ejemplo: Alumno.java — callback @PrePersist

```
// esto se ejecuta automáticamente justo antes de guardar en base de datos
@PrePersist
public void prePersist() {
    if (fechaInscripcion == null) {
        fechaInscripcion = LocalDate.now(); // pongo la fecha de hoy si no se especifica
    }
    if (estado == null) estado = Estado.ACTIVO;
}
```

7.3 Repositorios

Los repositorios extienden `JpaRepository` y añaden las consultas personalizadas necesarias mediante convención de nombres de Spring Data.

```
public interface CursoRepository extends JpaRepository<Curso, Long> {  
  
    List<Curso> findByEstado(Curso.Estado estado);  
  
    // para cuando borro un profesor y tengo que desasignarle los cursos  
    List<Curso> findByProfesorId(Long profesorId);  
}
```

7.4 Controladores REST

AuthController — login

El endpoint de login usa `AuthenticationManager` de Spring Security para validar las credenciales. Si son correctas, genera un token JWT con el nombre de usuario y su rol:

```
public class AuthController {  
    @Autowired  
    private AuthenticationManager authenticationManager;  
  
    @Autowired  
    private JwtUtil jwtUtil;  
  
    @Autowired  
    private UsuarioRepository usuarioRepository;  
  
    @PostMapping("/login")  
    public ResponseEntity<RespuestaLogin> iniciarSesion(@RequestBody PeticionLogin request) {  
        // si las credenciales no son correctas Spring lanza una excepción sola  
        Authentication auth = authenticationManager.authenticate(  
            new UsernamePasswordAuthenticationToken(request.getUsername(), request.getPassword())  
        );  
  
        Usuario usuario = usuarioRepository.findByUsername(auth.getName()).orElseThrow();  
        String token = jwtUtil.generarToken(usuario.getUsername(), usuario.getRol().name());  
  
        return ResponseEntity.ok(new RespuestaLogin(token, usuario.getUsername(), usuario.getRol().name()));  
    }  
}
```


MatriculaController — control de plazas

```
// compruebo si hay plazas libres
Curso curso = cursoRepository.findById(cursoId).orElse(null);
if (curso != null && curso.getCapacidad() != null) {
    List<Matricula> lista = matriculaRepository.findByCursoId(cursoId);
    int inscritos = lista.size();
    if (inscritos >= curso.getCapacidad()) {
        // uso 409 para que el frontend lo distinga de otros errores
        return ResponseEntity.status(status: 409).body(body: "CURSO_COMPLETO");
    }
}

return ResponseEntity.ok(matriculaRepository.save(matricula));
}
```

AlumnoController — borrado con cascada

```
// tuve que añadir el @Transactional porque sin él petaba al borrar si tenía matrículas
@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
@Transactional
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    if (!alumnoRepository.existsById(id)) return ResponseEntity.notFound().build();
    pagoRepository.deleteByAlumnoId(id);
    matriculaRepository.deleteByAlumnoId(id);
    alumnoRepository.deleteById(id);
    return ResponseEntity.noContent().build();
}
}
```

7.5 Endpoints de la API

Método	URL	Descripción	Roles
POST	/api/auth/login	Login, devuelve JWT	Público
GET	/api/alumnos	Lista todos los alumnos	ADMIN, PROFESOR
GET	/api/alumnos/{id}	Obtener alumno por id	ADMIN, PROFESOR, ALUMNO
POST	/api/alumnos	Crear alumno y genera usuario	ADMIN
PUT	/api/alumnos/{id}	Modificar alumno	ADMIN
DELETE	/api/alumnos/{id}	Borrar alumno y su usuario vinculado	ADMIN
GET	/api/profesores	Lista profesores	ADMIN, PROFESOR
POST	/api/profesores	Crear profesor	ADMIN
PUT	/api/profesores/{id}	Modificar profesor	ADMIN
DELETE	/api/profesores/{id}	Borrar profesor	ADMIN
GET	/api/cursos	Lista cursos	Autenticado
GET	/api/cursos/{id}	Obtener curso por id	Autenticado
POST	/api/cursos	Crear curso	ADMIN
PUT	/api/cursos/{id}	Modificar curso	ADMIN
DELETE	/api/cursos/{id}	Borrar curso	ADMIN
GET	/api/matriculas	Lista todas las matrículas	ADMIN, PROFESOR
GET	/api/matriculas/alumno/{id}	Matrículas de un alumno	Autenticado
GET	/api/matriculas/curso/{id}	Matrículas de un curso	ADMIN, PROFESOR
POST	/api/matriculas	Crear matrícula (control aforo)	ADMIN
PUT	/api/matriculas/{id}	Modificar matrícula	ADMIN
DELETE	/api/matriculas/{id}	Borrar matrícula	ADMIN
GET	/api/pagos	Lista todos los pagos	ADMIN
GET	/api/pagos/{id}	Obtener pago por id	ADMIN
GET	/api/pagos/alumno/{id}	Pagos de un alumno	Autenticado
POST	/api/pagos	Crear pago	ADMIN
PUT	/api/pagos/{id}	Modificar pago	ADMIN
DELETE	/api/pagos/{id}	Borrar pago	ADMIN
GET	/api/admin/usuarios	Lista usuarios	ADMIN
POST	/api/admin/usuarios	Crear usuario	ADMIN

GET	/api/alumnos/me	Devuelve el perfil del alumno autenticado a partir del token JWT	ALUMNO
DELETE	/api/admin/usuarios/{id}	Borrar usuario	ADMIN

7.6 Manejo global de excepciones

La clase Global Except Handler captura las excepciones de integridad de datos antes de que lleguen al cliente, evitando exponer el registro completo de la cadena de llamadas a métodos que estaban activas en ese momento hasta llegar al punto donde se produjo el error.

```
// pillo aquí los errores de base de datos para no devolver el stack trace al frontend
@RestControllerAdvice
public class GlobalExceptionHandler {    Can't initialize javac processor due to (most likely) a class loader p

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<String> handleDataIntegrity(DataIntegrityViolationException ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body(body: "Ya existe un registro con esos datos");
    }
}
```

8. Implementación del frontend

8.1 Estructura y organización

El frontend está compuesto únicamente por archivos HTML, CSS y JavaScript. No se utiliza ningún framework ni librería externa, ya que permite comprender con detalle cómo funciona el DOM y la comunicación con APIs REST.

La organización sigue el principio de un módulo por funcionalidad: cada página tiene su propio fichero .js y todas comparten las utilidades definidas en funciones.js..

8.2 funciones.js — librería compartida

Define la URL base de la API y las funciones comunes: llamarApi (wrapper de fetch que inyecta el token JWT en cada petición y gestiona errores 401 y 204), mostrarExito, verificarAcceso y construirMenu.

```
// llamada genérica al backend, la uso desde todas las páginas para no repetir el fetch
async function llamarApi(path, opts) {
  var metodo = opts ? opts.method || 'GET' : 'GET';
  var cuerpo = opts ? opts.body : null;
  var tok = obtenerToken();

  var cabeceras = { 'Content-Type': 'application/json' };
  if (tok) cabeceras['Authorization'] = 'Bearer ' + tok;

  var res = await fetch(API + path, {
    method: metodo,
    headers: cabeceras,
    body: cuerpo || null
  });

  if (res.status === 401) { cerrarSesion(); return null; } // token caducado
  if (!res.ok) {
    var msg = await res.text();
    throw new Error(msg || 'Error en la petición');
  }
  if (res.status === 204) return null;
  return res.json();
}
```

8.3 Autenticación y sesión

Cuando el usuario hace login, el frontend almacena el token JWT y los datos del usuario en Local Storage. En cada carga de página (excepto la de login), se verifica que el token existe y que el rol del usuario es el adecuado para esa página

```
const API = 'http://127.0.0.1:8082/api';

function obtenerToken() { return localStorage.getItem('token'); }
function obtenerUsuario() { return JSON.parse(localStorage.getItem('user')) || 'null'; }
```

8.4 Menú dinámico por rol

El menú de navegación se construye dinámicamente según el rol del usuario autenticado, mostrando solo las opciones que le corresponden:

- **ADMIN:** Dashboard, Alumnos, Profesores, Cursos, Matrículas, Pagos, Administración
- **PROFESOR:** Dashboard, Alumnos, Cursos, Matrículas
- **ALUMNO:** Mi perfil (vista de lectura de su propia informacion)

```
function verificarAcceso() {
  if (!obtenerToken()) {
    window.location.href = '/index.html';
    return false;
  }
  return true;
}

// compruebo que el usuario tenga uno de los roles que se le pasan
function verificarRol(...roles) {
  var user = obtenerUsuario();
  if (!user || !roles.includes(user.rol)) {
    window.location.href = '/pages/dashboard.html';
    return false;
  }
  return true;
}
```

8.5 Módulos JavaScript por funcionalidad

alumnos.js — búsqueda en local

Para evitar llamadas innecesarias al backend, los alumnos se guardan en un array local tras la carga inicial. La búsqueda filtra sobre ese arrays

```
var todosAlumnos = [];  
  
// filtro en local para no hacer una petición al backend cada vez que escribe algo  
function buscar(texto) {  
  if (!texto) {  
    mostrarAlumnos(todosAlumnos);  
    return;  
  }  
  var filtrados = [];  
  for (var i = 0; i < todosAlumnos.length; i++) {  
    var nombre = todosAlumnos[i].nombre + ' ' + todosAlumnos[i].apellidos;  
    if (nombre.toLowerCase().includes(texto.toLowerCase())) {  
      filtrados.push(todosAlumnos[i]);  
    }  
  }  
  mostrarAlumnos(filtrados);  
}
```

matricula.js — distinción de error 409

El backend devuelve “CURSO_COMPLETO” como cuerpo del error 409 cuando no hay plazas. El frontend lo detecta y muestra un modal específico:

```
} catch (err) {  
  // el backend devuelve CURSO_COMPLETO cuando no hay plazas  
  if (err.message.includes('CURSO_COMPLETO')) {  
    cerrarModal('modalMatricula');  
    abrirModal('modalCursoCompleto');  
  } else {  
    alert('Error: ' + err.message);  
  }  
}
```

admin.js — estadísticas de pagos

```
// separo los pagos por estado
for (let i = 0; i < pagos.length; i++) {
  const p = pagos[i];
  if (p.estado === 'PENDIENTE') pendientes.push(p);
  | else if (p.estado === 'PAGADO') pagados.push(p);
  | else if (p.estado === 'VENCIDO') vencidos.push(p);
  | else if (p.estado === 'CANCELADO') cancelados.push(p);
}

let totalRecaudado = 0;
for (let i = 0; i < pagados.length; i++) {
  | totalRecaudado += pagados[i].importe || 0;
}

let totalPendiente = 0;
for (let i = 0; i < pendientes.length; i++) {
  | totalPendiente += pendientes[i].importe || 0;
}
```

admin.js — ocupación de cursos

```
// tabla con cuántos alumnos tiene cada curso y el porcentaje de ocupación
async function cargarOcupacion() {
  const cursos = await llamarApi('/cursos');
  const matriculas = await llamarApi('/matriculas');
  const tbody = document.getElementById('tbodyOcupacion');

  if (!cursos || cursos.length === 0) {
    tbody.innerHTML = '<tr class="empty-row"><td colspan="5">No hay cursos</td></tr>';
    return;
  }

  // cuento cuántos alumnos tiene cada curso mirando las matrículas
  const countPorCurso = {};
  if (matriculas) {
    for (let i = 0; i < matriculas.length; i++) {
      const m = matriculas[i];
      if (m.curso && m.curso.id) {
        if (countPorCurso[m.curso.id] === undefined) {
          countPorCurso[m.curso.id] = 0;
        }
        countPorCurso[m.curso.id] = countPorCurso[m.curso.id] + 1;
      }
    }
  }
}
```

8.6 Interfaz de usuario (CSS)

El diseño visual está desarrollado en CSS puro mediante un sistema de componentes reutilizables. La paleta usa el azul como color principal, combinado con tonos neutros para fondos y texto, garantizando un contraste adecuado en toda la interfaz.

Para mostrar el rol y el estado de cada usuario se utilizan badges o etiquetas de color en las vistas de listado: azul para el administrador, verde para el profesor y naranja para el alumno. En cuanto al estado, el verde indica cuenta activa, el rojo una baja y el naranja una situación pendiente.

Los modales siguen siempre la misma estructura: una superposición semitransparente sobre el contenido y una caja blanca centrada con cabecera, cuerpo y pie de botones. Las tablas incluyen filas alternas y cabecera en azul oscuro para mejorar la lectura de los datos.

Las notificaciones se gestionan con un toast verde en la esquina inferior derecha que desaparece solo tras 2,5 segundos. El diseño es además responsive: en pantallas menores de 640 píxeles, los formularios pasan de cuadrícula a una sola columna.

9. Seguridad

9.1 Autenticación con JWT

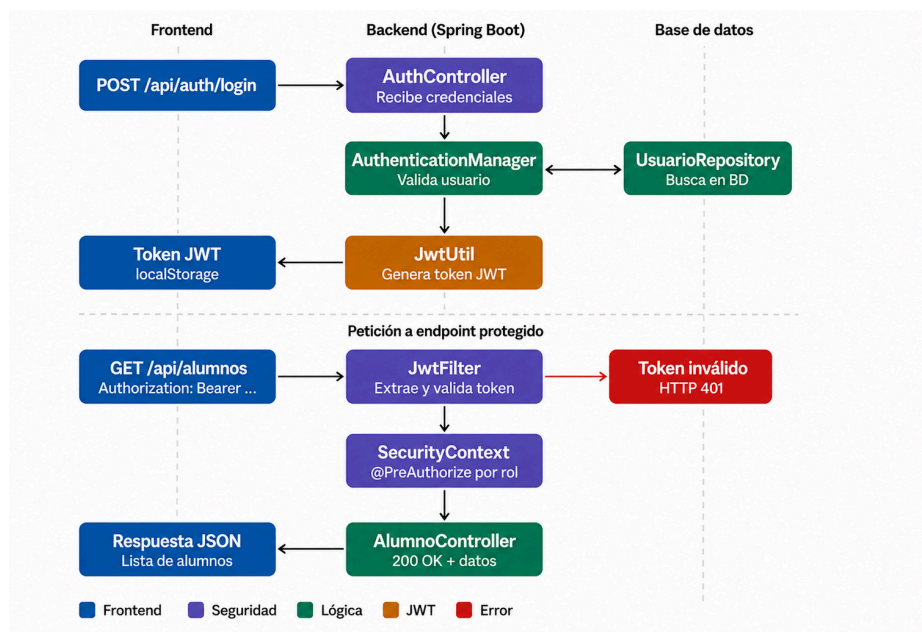
El sistema emplea JWT (JSON Web Tokens) para gestionar la autenticación de forma stateless. Tras el login, el usuario recibe un token que debe incluir en la cabecera HTTP de todas las peticiones posteriores.

Cada token contiene los siguientes claims:

- campo sub con el nombre de usuario
- rol asignado (ADMIN, PROFESOR o ALUMNO)
- fecha de emisión (iat)
- fecha de expiración (exp) : establecida 24 horas después de la emisión.

La firma se genera con el algoritmo HMAC-SHA256 a partir de una clave secreta configurada en el servidor. Sin dicha clave es imposible forjar un token válido, lo que garantiza la integridad y autenticidad de cada petición.

No obstante, una vulnerabilidad conocida es que el token se almacena actualmente en localStorage del navegador. Esto es funcional para desarrollo, pero introduce una superficie de ataque ante XSS (un script malicioso inyectado en la página podría leer el token y suplantar al usuario). Por ello si se llevase a producción sería necesario migrar el almacenamiento a cookies HttpOnly, que no son accesibles desde JavaScript y eliminan este vector de ataque.



9.2 Filtro JWT (JwtFilter)

El filtro JwtFilter intercepta todas las peticiones HTTP antes de que lleguen a los controladores. Extrae el token del encabezado, lo valida y, si es correcto, establece el contexto de seguridad de Spring:

```
public class JwtFilter extends OncePerRequestFilter {  
    private final JwtUtil jwtUtil;  
  
    @Override  
    protected void doFilterInternal(HttpServletRequest request,  
                                    HttpServletResponse response,  
                                    FilterChain filterChain) throws ServletException, IOException {  
        String header = request.getHeader("Authorization");  
  
        if (header != null && header.startsWith("Bearer ")) {  
            String token = header.substring(7); // quito el "Bearer " del principio  
            if (jwtUtil.esValido(token)) {  
                String username = jwtUtil.obtenerUsername(token);  
                String rol = jwtUtil.obtenerRol(token);  
                var auth = new UsernamePasswordAuthenticationToken(  
                    username, null,  
                    List.of(new SimpleGrantedAuthority("ROLE_" + rol))  
                );  
                SecurityContextHolder.getContext().setAuthentication(auth);  
            }  
        }  
  
        filterChain.doFilter(request, response);  
    }  
}
```

9.3 Configuración de Spring Security

La clase SecurityConfig se encarga de gestionar la seguridad de la aplicación de forma centralizada. En ella se indica qué rutas son accesibles sin necesidad de autenticación y cuáles requieren un rol específico.

Las rutas de autenticación (/api/auth/) son públicas porque cualquier usuario tiene que poder hacer login antes de tener un token. Lo mismo ocurre con la documentación de Swagger (/swagger-ui/ y /v3/api-docs/), que se deja abierta para poder consultar la API sin restricciones. La sección de administración (/api/admin/) está protegida y solo pueden acceder a ella los usuarios con rol ADMIN. El resto de endpoints simplemente exigen que el usuario haya iniciado sesión, sin importar el rol que tenga.

El control más fino, es decir, qué rol puede crear, modificar o solo consultar cada recurso, se gestiona directamente en cada controller usando la anotación @PreAuthorize

```
@Bean  
public SecurityFilterChain seguridad(HttpSecurity http) throws Exception {  
    http  
        .cors(cors -> cors.configurationSource(configurarCors()))  
        .csrf(csrf -> csrf.disable())  
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))  
        .authorizeHttpRequests(auth -> auth  
            .requestMatchers(...patterns: "/api/auth/**").permitAll() // login sin token  
            .requestMatchers(...patterns: "/swagger-ui/**", "/v3/api-docs/**", "/swagger-ui.html").permitAll()  
            .requestMatchers(...patterns: "/api/admin/**").hasRole(role: "ADMIN")  
            .requestMatchers(...patterns: "/api/profesor/**").hasAnyRole(...roles: "ADMIN", "PROFESOR")  
            .anyRequest().authenticated()  
        )  
        .addFilterBefore(jwtFilter, beforeFilter: UsernamePasswordAuthenticationFilter.class);  
    return http.build();  
}
```

9.4 Hashing de contraseñas

Las contraseñas nunca se guardan en texto plano. Se usa BCrypt con factor de coste 10 (el valor por defecto de Spring Security), lo que hace computacionalmente inviable un ataque de fuerza bruta:

```
@Bean
public PasswordEncoder codificadorClave() {
    return new BCryptPasswordEncoder();
}
```

9.5 CORS

Dado que el frontend y el backend están en puertos distintos (3000 y 8082), es necesario configurar CORS para que el navegador permita las peticiones cross-origin:

```
// sin esto el navegador bloqueaba todas las peticiones del frontends

@Bean
public CorsConfigurationSource configurarCors() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOriginPatterns(List.of("*"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(allowCredentials: true);
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration(pattern: "**", config);
    return source;
}
```

9.6 Validaciones de integridad

Validación	Capa	Mecanismo
DNI único (alumnos y profesores)	BD + Backend	Constraint UNIQUE + GlobalExceptionHandler devuelve 409
Teléfono único	BD + Backend	Constraint UNIQUE + GlobalExceptionHandler devuelve 409
Matrícula duplicada	BD + Backend	Constraint UNIQUE(alumno_id, curso_id)
Precio no negativo	Backend	Validación manual en el controller
Capacidad de curso	Backend	Conteo de matrículas antes de insertar
Borrado explícito en transaccion	Backend	Llamadas explícitas a los repositorios en @Transactional

10. Pruebas

10.1 Estrategia de pruebas

Se han implementado pruebas unitarias con JUnit 5 para los componentes más relevantes del sistema. Las pruebas se encuentran en el directorio `backend/src/test/java/com/academia/`

10.2 AlumnoTest.java

Lleva a cabo 3 test distintos los cuales son:

- **testDatosBasicos** (nombre, apellidos y dni se asignan correctamente),
- **testEstado** (el estado ACTIVO se guarda bien)
- **testCamposNulos** (dni y teléfono son null por defecto).

Nota de **testCamposNulos**: con la incorporación de DNI como campo obligatorio, este test ha sido revisado para reflejar que un Alumno válido debe tener DNI no nulo antes de ser persistido. Por ello, en este test el DNI se establece explícitamente a null para verificar que la validación rechaza correctamente el objeto.

10.3 JwtUtilTest.java

Los tests desarrollados para validar el comportamiento del servicio JWT son los siguientes:

- **testGenerarToken**: comprueba que el token generado tras el login no es nulo.
- **testUsernameEnToken**: verifica que el método *"obtenerUsername"* extrae correctamente el nombre de usuario almacenado en el token.
- **testRolEnToken**: confirma que el método *"obtenerRol"* devuelve el rol asociado al usuario de forma correcta.
- **testTokenValidoEInvalido**: valida que el método *"esValido"* retorna true ante un token legítimo y false ante una cadena inválida.

10.4 Pruebas manuales con Postman

Además de los tests automáticos, se han realizado pruebas manuales con Postman para validar el comportamiento de cada endpoint:

Prueba	Resultado esperado	Resultado obtenido
Login con credenciales correctas	200 OK + JWT	OK
Login con contraseña incorrecta	401 Unauthorized	OK
Petición sin token	401 Unauthorized	OK
Crear alumno con DNI duplicado	409 Conflict	OK
Matricular en curso lleno	409 + CURSO_COMPLETO	OK
Borrar alumno con matrículas	204, cascada correcta	OK
Crear curso con precio negativo	400 Bad Request	OK
Acceso a /api/admin como PROFESOR	403 Forbidden	OK
Crear alumno → comprobar usuario generado en BD	200 OK	OK
Login alumno con credenciales DNI/DNI → GET /api/alumnos/me	200 OK	OK
GET /api/alumnos/me con token de ADMIN	403 Forbidden	OK

10.5 Ejecución de los tests

Los tests se ejecutan con Maven Surefire Plugin y generan un informe en target/surefire-reports/.. Todos los tests deben pasar antes de hacer un commit en la rama principal.

```
cd backend
mvn test
```

11. Manual de usuario

11.1 Acceso al sistema

Para acceder a la aplicación, abrir un navegador e ir a la dirección donde esté servido el frontend (<http://localhost:3000>). Se mostrará la pantalla de login.

Introducir el nombre de usuario y la contraseña, luego pulsar **Entrar**. Si las credenciales son correctas, se redirige al panel principal. Si son incorrectas, aparece un mensaje de error en rojo.

11.2 Panel principal (Dashboard) (ADMIN)

Al iniciar sesión, el usuario ve el panel principal adaptado a su rol:

- **Admin:** tarjetas con el total de alumnos, profesores, cursos y pagos, más accesos directos a todas las secciones.
- **Profesor:** saludo personalizado y tarjetas de acceso a alumnos, cursos y matrículas.
- **Alumno:** actualmente con un único apartado para consultar la información del alumno.

11.3 Gestión de alumnos(ADMIN)

1. Hacer clic en **Alumnos** en el menú de navegación.
2. La tabla muestra todos los alumnos registrados con sus datos principales.
3. Para buscar, escribir en el campo de búsqueda (filtra por nombre o apellidos en tiempo real).
4. Para crear un alumno, pulsar **Nuevo alumno**, rellenar el formulario y guardar.
5. Para editar, pulsar el botón **Editar** de la fila correspondiente.
6. Para borrar, pulsar **Borrar** y confirmar. Se eliminan también sus matrículas y pagos.

Nota: Al borrar un alumno se eliminan también todas sus matrículas y pagos. Esta acción no se puede deshacer. Además estas acciones afectan la cuenta del usuario

11.4 Gestión de profesores (ADMIN)

El funcionamiento es idéntico al de alumnos. La diferencia principal es el campo de **especialidad**. Y que al borrar un profesor, sus cursos no se eliminan pero pierden la asignación del profesor.

11.5 Gestión de cursos (ADMIN)

1. Ir a **Cursos** en el menú.
2. Pulsa **Nuevo curso** para crear uno.
3. Selecciona el profesor del desplegable.
4. Indica la capacidad máxima
5. El precio debe ser mayor o igual a cero.
6. El estado puede ser: ACTIVO, INACTIVO o COMPLETO.

11.6 Matrículas (ADMIN)

1. Ir a **Matrículas**.
2. Pulsa **Nueva matrícula**.
3. Selecciona el alumno y el curso en los desplegables.
4. Si el curso está lleno, aparece un aviso y la matrícula no se crea.
5. Si el alumno ya está matriculado en ese curso, el sistema mostrará un error.

11.7 Pagos(ADMIN)

1. Ir a **Pagos**.
2. Usa los botones de filtro (TODOS, PENDIENTE, VENCIDO, PAGADO, CANCELADO) para ver solo los pagos de un estado.
3. Para registrar un pago, pulsar **Registrar pago**.
4. El campo "Curso" es opcional.
5. Para actualizar el estado de un pago existente, usar el botón **Editar**.

11.8 Panel de administración (ADMIN)

Solo accesible para usuarios con rol ADMIN. Muestra:

- **Resumen de pagos:** total recaudado, número de pendientes, vencidos y cancelados.
- **Alumnos con pagos vencidos:** tabla con los pagos que han superado su fecha de vencimiento.
- **Ocupación de cursos:** barra visual de porcentaje de ocupación por curso (verde / naranja / rojo).
- **Gestión de credenciales:** creación y borrado de cuentas de usuario del sistema.

Nota : Un administrador no puede borrar su propia cuenta desde este panel.

11.9 Cerrar sesión (ADMIN)

Pulsar el botón **Salir** del menú de navegación, después dale al botón de confirmar y el token se elimina del almacenamiento local y el usuario es redirigido a la página de login.

Nota: Todo lo anteriormente relacionado es con el **ROL= ADMINISTRADOR**

11.10 Panel Profesor - Rol Profesor

Solo accesible para usuarios con rol PROFESOR (también el ADMIN puede consultar las mismas secciones). Al iniciar sesión se muestra:

- Saludo personalizado con el nombre de usuario.
- Tarjetas resumen: Total Alumnos, Total Cursos y Matrículas registradas en el sistema.
- Accesos rápidos a las secciones disponibles para el profesor: Alumnos, Cursos y Matrículas.

Permisos del profesor

El profesor tiene acceso de consulta a la información del centro, pero no puede crear, modificar ni eliminar registros. En concreto:

- **Alumnos:** puede ver el listado y los datos de cada alumno, pero los botones de alta, edición y borrado solo están disponibles para el administrador.

- **Cursos:** puede consultar el catálogo de cursos, su profesor asignado, capacidad, precio y estado. La gestión (crear, editar, borrar) está reservada al administrador.
- **Matrículas:** puede ver todas las matrículas, tanto el listado general como las matrículas por curso. La creación y borrado de matrículas se realizan desde el panel del administrador.

11.11 Panel Alumno

Accesible para usuarios con rol ALUMNO. La cuenta del alumno se crea automáticamente cuando el administrador da de alta su ficha: el nombre de usuario y la contraseña inicial coinciden con su DNI. Se recomienda al alumno cambiar la contraseña la primera vez que acceda.

Al iniciar sesión se muestra:

- Saludo personalizado con el nombre del alumno.
- Una única tarjeta de acceso: Mi perfil.

Mi perfil

Pulsando la tarjeta Mi perfil (o el enlace del menú de navegación) se accede a una vista de solo lectura con los datos personales del alumno: nombre, apellidos, DNI, email, teléfono, dirección, fecha de nacimiento, fecha de inscripción y estado.

El alumno no puede modificar sus datos directamente. Si alguno de los campos no es correcto debe avisar a la administración para que lo actualice desde el panel correspondiente.

12. Despliegue

12.1 Requisitos del entorno

Componente	Requisito
Backend	Java 21, Maven 3.9+
Base de datos	PostgreSQL 14+ (o cuenta en Neon)
Frontend	Servidor HTTP
Red	Puerto 8082 abierto para el backend

12.2 Configuración

El fichero `application.properties` contiene la configuración de la aplicación:

```
spring.application.name=academia

# PostgreSQL - Neon
spring.datasource.url=jdbc:postgresql://ep-steep-pine-adxuimcp-pooler.c-2.us-east-1.aws.neon.tech
spring.datasource.username=neondb_owner
spring.datasource.password=npg_NgWdeJBec4V1
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.properties.hibernate.format_sql=true

# JWT
app.jwt.secret=academia-secret-key-2024-must-be-at-least-256-bits-long-for-hmac-sha
app.jwt.expiration=86400000

# CORS
spring.web.cors.allowed-origins=*

server.port=8082
```

Nota : Las credenciales de la base de datos y la clave JWT deben configurarse con variables de entorno en producción. Nunca incluir contraseñas en el código fuente.

12.3 Arrancar el backend

Desde el directorio backend, y ejecutando:

```
mvn spring-boot:run
```

O generando un JAR ejecutable:

```
mvn package -DskipTests  
java -jar target/academia-1.0.0.jar
```

12.4 Arrancar el frontend

Desde el directorio frontend, ejecutando:

```
python3 -m http.server 3000.
```

Y luego abrir <http://localhost:3000> en el navegador.

12.5 Base de datos en Neon

El proyecto usa Neon como base de datos PostgreSQL en la nube. Neon ofrece un plan gratuito suficiente para desarrollo y pruebas. Hibernate crea y actualiza el esquema automáticamente al arrancar la aplicación (no es necesario ejecutar scripts SQL manualmente), por la propiedad **ddl-auto=update**.

12.6 Documentación de la API con Swagger

Una vez arrancado el backend, la documentación de todos los endpoints está disponible en:

<http://localhost:8082/swagger-ui.html>

Desde ahí se pueden probar los endpoints directamente desde el navegador, lo que facilita las pruebas durante el desarrollo.

12.7 Usuarios de Prueba

Usuario	Contraseña	Rol
admin	admin123	ADMIN
profesor1	prof123	PROFESOR
alumno1	alum123	ALUMNO

Nota: Los usuarios de profesores deberán crearse por un administrador, dentro del panel de Administración, sin embargo los alumnos no necesitan ser creados manualmente como usuarios. Al darlos de alta en Gestión de alumnos → Nuevo alumno, su cuenta se genera automáticamente con username=DNI y contraseña inicial=DNI.

13. Conclusiones y trabajo futuro

13.1 Conclusiones

El desarrollo de este proyecto ha permitido poner en práctica de forma integrada los conocimientos adquiridos a lo largo del ciclo formativo de DAM: programación orientada a objetos con Java, acceso a bases de datos mediante JPA, desarrollo de APIs REST, y desarrollo web con HTML, CSS y JavaScript, además de permitirme aprender cosas nuevas como seguridad con JWT.

El resultado es una aplicación funcional que resuelve un problema real: la gestión digitalizada de una academia de formación. Cumpliendo todos los objetivos planteados al inicio del proyecto, incluyendo la autenticación con JWT, el control de acceso por roles, las validaciones de negocio y la interfaz de usuario adaptada a cada tipo de usuario.

Algunos de los aprendizajes más destacados durante el desarrollo:

- La configuración de Spring Security es compleja al principio, pero una vez entendida resulta muy potente y flexible.
- El manejo de CORS requiere atención especial cuando el frontend y el backend están en puertos distintos.
- La importancia de las transacciones (`@Transactional`) en operaciones que implican múltiples cambios en la base de datos.
- Separar las responsabilidades en módulos JavaScript facilita el mantenimiento del código frontend.
- La anotación `@UniqueConstraint` a nivel de tabla (para restricciones sobre varios campos) es la forma correcta de prevenir duplicados compuestos como el de matrícula.
- La relación `@OneToOne` con `CascadeType.ALL` permite gestionar la creación y eliminación del usuario vinculado al alumno de forma transparente, simplificando la lógica del controller

13.2 Dificultades encontradas

Durante el desarrollo surgieron varios problemas que requirieron investigación y resolución:

- **CORS:** El navegador bloqueaba las peticiones del frontend al backend. Sin embargo, al configurar correctamente CorsConfigurationSource en Spring Security se solucionó
- **Borrado en cascada:** Hibernate lanzaba excepción de integridad. Pero con el uso de @Transactional y borrado explícito en orden correcto.
- **JWT y longitud de clave:** JWT requiere claves de al menos 256 bits para HMAC-SHA256.
- **Null vs. string vacío:** El navegador enviaba campos vacíos como string vacío, por lo que fue necesario convertir explícitamente a null antes de enviar.
- **Vínculo automático Usuario-Alumno:** fue necesario asegurarse de que el usuario se persistía junto con el alumno en la misma transacción, y de que el borrado en cascada también eliminaba el usuario sin lanzar excepciones de integridad referencial.

13.3 Trabajo futuro

Hay funcionalidades que quedan pendientes para futuras versiones del sistema:

- **Panel del alumno (prioritario):** Es la funcionalidad más relevante pendiente a ampliar. Actualmente solo muestra la información del alumno. Se planea añadir consulta de matrículas activas y pagos propios del alumno autenticado.
- **Capa de servicios:** Extraer la lógica de negocio de los controllers a una capa service, siguiendo una arquitectura más limpia y testeable.
- **Seguridad del token:** Migrar el almacenamiento del JWT de localStorage a cookies HttpOnly para eliminar el riesgo de XSS.
- **Búsqueda de pagos por fechas:** Existe un TODO en PagoRepository para filtrado por rango de fechas.
- **Exportación a PDF/Excel:** Descarga de listados de alumnos, matrículas o pagos.
- **Notificaciones:** Envío de correos automáticos para avisar de pagos próximos a vencer.
- **Tests de integración:** Ampliar cobertura con tests que prueben los endpoints completos.
- **Dockerización:** Empaquetar backend y base de datos en contenedores Docker para facilitar el despliegue.

14. Bibliografía

Bibliografía

- Baeldung. (2024). *REST with Spring Tutorial*.
<https://www.baeldung.com/rest-with-spring-series>
- Baeldung. (2024). *Spring Security JWT Authentication*.
<https://www.baeldung.com/spring-security-oauth-jwt>
- Baeldung. (2024). *Java JSON Web Tokens with JJWT*.
<https://www.baeldung.com/java-json-web-tokens-jjwt>
- CertiDevs. (2024). *Aprender Spring Boot: introducción, instalación y entorno*.
<https://certidevs.com/aprender-spring-boot-introduccion-instalacion-entorno>
- El Blog del Programador. (2024). *Cómo implementar autorización y autenticación JWT en Java Spring Boot*.
<https://elblogdelprogramador.com/posts/como-implementar-autorizacion-y-autenticacion-jwt-en-java-spring-boot/>
- Hibernate Documentation. (2024). *Hibernate ORM 6.x User Guide*.
https://docs.jboss.org/hibernate/orm/6.0/userguide/html_single/
- Horstmann, C. S. (2022). *Core Java, Volume I: Fundamentals* (12.ª ed.). Pearson.
- JJWT Documentation. (2024). *JJWT — Java JWT: JSON Web Token for Java and Android*.
<https://github.com/jwt/jjwt>
- JWT.io. (2024). *Introduction to JSON Web Tokens*. <https://www.jwt.io/introduction>
- Mozilla Developer Network. (2024). *MDN Web Docs — JavaScript Reference*.
<https://developer.mozilla.org/es/docs/Web/JavaScript>
- Neon Documentation. (2024). *Neon Serverless Postgres*. <https://neon.tech/docs>
- OpenAPI Initiative. (2024). *OpenAPI Specification*. <https://spec.openapis.org/oas/latest.html>
- Spring Framework. (2024). *Spring Boot — Official Project Page*.
<https://spring.io/projects/spring-boot>
- Spring Framework. (2024). *Spring Data JPA Reference Documentation*.
<https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- Spring Framework. (2024). *Spring Security — Official Project Page*.
<https://spring.io/projects/spring-security>
- Spring Framework. (2024). *Spring Security Reference*.
<https://docs.spring.io/spring-security/reference/>

- Walls, C. (2022). *Spring Boot in Action* (2.^a ed.). Manning Publications.
- W3Schools. (2024). *CSS Tutorial*. <https://www.w3schools.com/Css/>
- W3Schools. (2024). *HTML Tutorial*. <https://www.w3schools.com/html/>

Videografía

- DavinchiCoder. (2024). *SPRING SECURITY con JWT DESDE CERO: PROTEGE tu aplicación FACILMENTE con JAVA!* [Vídeo]. YouTube. https://youtu.be/okp_l4NGPZ8
- ManuDev. (2024). *CREA UN FORMULARIO DE INICIO DE SESION CON HTML Y CSS | Fácil y Rápido!* [Vídeo]. YouTube. <https://youtu.be/XPrs9nqiYOc>
- Programando en JAVA. (2024). *JWT SPRING SECURITY con SPRING BOOT 3 - Tutorial Completo Fácil* [Vídeo]. YouTube. <https://youtu.be/-Z4a0bKr2Pg>
- Rojas Córscico, I. S. (2024). *¿Cómo crear el login? Spring Boot 3 + Spring Security 6 + JWT Authentication* [Vídeo]. YouTube. <https://youtu.be/nwqQYCM4YT8>
- TodoCode. (2024). *MASTER CLASS de SPRING BOOT (Introductoria)* [Vídeo]. YouTube. <https://youtu.be/8X2acANBuLk>
- Un Programador Nace. (2024). *Spring Security 6: La Guía Completa para Principiantes* [Vídeo]. YouTube. <https://youtu.be/IPWBQDMIYkc>

Nota: Además de la consultas realizadas a profesionales del sector en prácticas, apuntes de clase y preguntas a compañeros de clase
--